

Machine Learning I Handbook

Faculty of Computer Science

Economics and Data Science

Daniel Fabian Freire Serioguina

Contents

1. Introduction to ML	2
1.1. Basic definitions.	2
1.2. Types of tasks in machine learning.	2
1.3. Building an algorithm (hypothesis)	3
1.4. Filtering data	4
2. Linear models	5
2.1. Introducing linear models.	5
2.2. Usage of linear models.	5
2.3. Estimating errors in regression models	6
3. Linear Regression	11
3.1. Overfitting	11
3.2. Estimating the quality of models	11
3.3. Training a linear regression	11
4. Gradient descent	12
4.1. Gradient and its characteristics	12
4.2. Gradient descent	12
4.3. Evaluating a gradient	13
4.4. Modifications of the gradient descent	15
5. Linear classification	16
5.1. Training linear classifiers	16
5.2. Quality metrics of classification	17
5.3. Log-regression	19
5.4. Support vector machine	19
6. Multi-class classification and categorical features	20
6.1. Multi-class classification	20
6.2. Classification with overlapping classes	20
6.3. Categorical features	20
6.4. Seminar	21

1. Introduction to ML

1.1. Basic definitions.

Object (x).

The thing we are analysing.

Object set ($\mathbb{X}$).

The set of all the possible objects of a certain task. Euclidean space.

Target variable (y).

The answer to the task we are solving.

Target set ($\mathbb{Y}$).

The set of all the possible targets of a certain task. Euclidean space.

Training set ($X := (x_i, y_i)_i$).

A set of object for which there are **correct** targets, and $|X| = \ell$.

Datadrift.

We say we have encountered datadrift when the training set is no longer relevant for today or tomorrow.

features.

The characteristics of objects x , where d is the amount of features.

$$X_i = (x_{i1}, \dots, x_{id})$$

Numerical: $x_j \in \mathbb{R}$,

Categorical: $x_j \in \{C_1, \dots, C_m\}$, (Only supports equality comparison)

Ordinal: $x_j \in \{C_1, \dots, C_m\}$ (Supports ordering operations)

1.2. Types of tasks in machine learning.

The type of task in machine learning is defined by the target variable y from the target variable set $\mathbb{Y}$.

Supervised Learning.

The training set $X := (x_i, y_i)_i$ for which the targets are available.

1. Linear Regression ($\mathbb{Y} = \mathbb{R}$)

2. Classification ($|\mathbb{Y}| < \infty$) :

(Binary : $\mathbb{Y} = \{0, 1\}$, Multi-class : $\mathbb{Y} = \{1, \dots, k\}$, Multi-label: $\mathbb{Y} = \{0, 1\}^k$)

3. Ranking

Unsupervised learning.

We are given a training set $X_i := (x_i, y_i)_i$ without the target variables as answers.

1. Clustering: We try to break a set into subsets such that are similar to each other.
2. Density Estimation: (Search for anomalies)

Partial Learning A method that combines supervised and unsupervised learning.

1.3. Building an algorithm (hypothesis)

The **task** that we will be solving is:

$$a : \mathbb{X} \rightarrow \mathbb{Y}$$

Error function.

We need to be able to formally evaluate the quality of algorithms. The error function is used to judge the model's performance in a meaningful, interpretable way. Often, it is used to evaluate the entire dataset, and give real world utility.

As a popular example we have MSE (Mean Squared Error).

$$\frac{1}{\ell} \sum_{i=1}^{\ell} (a(x_i) - y_i)^2$$

The smaller the amount of the error function, the better the algorithm.

Loss function.

A loss function will compute the error for a single training example from the training set X . It measures how far off the model's **prediction** (a) is from the target variable y .

$$L : \mathbb{Y} \times \mathbb{Y} \rightarrow \mathbb{R}_+$$

Continuing with the example from before:

$$L(y, a) = (y - a)^2$$

Algorithm ($\mathcal{A}$)

Once we have fixed an error function, we are able to build the algorithm $a(x)$. Beforehand we fix a family of algorithms $\mathcal{A}$.

$$\mathcal{A} = \{a(x) = w_0 + w_1x_1 + \dots + w_dx_d \mid w_0, w_1, \dots, w_d \in \mathbb{R}\}, x_i : i \text{ feature of the object } x$$

$$a(x) = w_0 + w_1x_1 + \dots + w_dx_d = w_0 + \sum_{j=1}^d w_jx_{ij}$$

The best algorithm we are able to obtain from this is obtained through solving the following problem:

$$\frac{1}{\ell} \sum_{j=1}^{\ell} \left(w_0 + \sum_{j=1}^d w_j x_{ij} - y_i \right)^2 \rightarrow \min_{w_0, \dots, w_d}, x_{ij} \text{ is the } j \text{ feature of on an } i \text{ object}$$

The process of finding an optimal algorithm is called training a model (algorithm).

1.4. Filtering data

For an algorithm to be useful we need to be able to feed it good data. Data might need to be altered in order to be considered “good”.

1. Linear models:

Normalize them by subtracting the average,

and dividing by the Variance of all columns of the 'objects-features' matrix.

2. Outliers:

Objects that are incorrect examples, the presence of which ruins a model.

3. Noise:

Objects that do not have any relation to the target variable y.

Overfitting. Is the result of a model that corresponds to closely to the training set $X_i := (x_i, y_i)_i$ and fails to fit and / or predict future observations.

As an example:

$$a(x) = \begin{cases} y_i, & x = x_i \\ 0, & x \notin X \end{cases}$$

This would be an awful model, as it would not learn anything at all, as it follows the target variable way too closely.

2. Linear models

2.1. Introducing linear models.

Linear models.

We can describe the vast majority of linear regression models as the sum of features with certain weights.

$$a(x) := w_0 + \sum_{j=1}^d w_j x_j$$

The **parameters** of a model are its **weights (coefficients)** w_j , where w_0 is the bias (intercept).

Our form of presenting the linear model can be rewritten as:

$$a(x) := w_0 + \langle w, x \rangle, \quad w = (w_1, \dots, w_d)$$

If we add to each feature description of each object an $(d + 1)$ feature equal to one, it would could as the free coefficient, and would permit us to rewrite the linear model as:

$$a(x) := \langle w, x \rangle$$

2.2. Usage of linear models.

An important criteria to consider is that in the linear model each feature has a linear effect on the target variable, and this target variable does not depend from a combination of features.

Categorical features.

Suppose we have a categorical feature the value of which we cannot compare the size of, subtract or add from. These types of factors cannot be used in linear models without additional modification.

One-hot coding.

Essentially, one hot encoding is a method for converting categorical variables into a binary format, where we create new columns for each category where 1 is the category being present and 0 being absent.

Suppose that the categorical feature $f_{j(x)}$ takes in values from the set $C = \{c_1, \dots, c_m\}$. We replace this categorical feature $f_{j(x)}$ for the m binary factors $b_1(x), \dots, b_m(x)$, where each one is the indicator function of one of the possible categorical values.

$$b_i(x) = [f_{j(x)} = c_i], \quad \text{s.t. } b_1(x) + \dots + b_m(x) = 1 \text{ (lineary dependent)}$$

The fact that they are lineary dependent is a problem, we could prevent this by throwing out one of the binary features, but if the test set gains a new category, it might be encoded by a zero binary feature, and with the deletion of which it would loose meaning.

Suppose we are trying to make a model for estimating the price of apartments using a linear model, where the categorical feature is the location of the apartment building. After performing one-hot encoding we would obtain the following result:

$$a(x) = w_1[f(x) = c_1] + \dots + w_m[f(x) = c_m] + \{\text{relation with other features}\}$$

Which makes more sense than a standard linear model, as the location would define an “initial estimate” for the value of the apartment building, and only after all of the remaining features would further adjust it.

Working with texts (bag of words).

Let's borrow the same example from before, and suppose we are trying to predict its price via its textual description.

Find all of the words in our set: $\{c_1, \dots, c_m\}$

Format the text using m features: $b_1(x), \dots, b_m(x)$, s.t. $b_j(x) := \text{num of words } c_j \text{ in the text.}$

The linear model would look like: $a(x) = w_1 b_1(x) + \dots + w_m b_m(x) + \dots$

Essentially, each inclusion of the word c_j would change the predicted price by w_j .

Binarization of numerical features.

Suppose that for the previous example we are given the distance to the metro (x_j). It seems clear that there mustn't necessarily be a linear dependency between the feature and the target variable.

To solve this problem we can binarize the features.

For this we take a $\{t_1, \dots, t_m\}$ net, and add the points $t_0 = -\infty, t_{m+1} = +\infty$.

Set up new features as: $b_{i(x)} = [t_{i-1} < x_j \leq t_i]$, $i = 1, \dots, m+1$

The linear model would look like: $a(x) = w_1 [t_0 < x_j \leq t_1] + \dots + w_{m+1} [t_m < x_j \leq t_{m+1}]$

Thus we have considered the non linear dependency between the feature and the target variable when predicting the price of the apartment for each interval of distance from the nearest metro.

2.3. Estimating errors in regression models

Previously we've spoken that there is a need to properly evaluate the quality of the predictions given by a model. Suppose that a is the model prediction, and y is our target variable. Here we will look through several methods of evaluating the deviation of the prediction from the original answer, that we will call $L(y, a)$.

MSE and R^2 .

The main way to evaluate the deviation of the prediction (answer) of the model a , and the target variable y .

$$L(y, a) = (a - y)^2$$

Mean Squared Error (MSE)

The main error function based on the squared difference.

$$\text{MSE}(a, X) := \frac{1}{\ell} \sum_{i=1}^{\ell} (a(x_i) - y_i)^2$$

As with all metrics, this error function is not perfect, as its results are not great for interpretation because it does not keep the unit of measurement of the original target variable. To avoid this problem, the Root mean squared error (RMSE) is introduced.

$$\text{RMSE}(a, X) := \sqrt{\frac{1}{\ell} \sum_{i=1}^{\ell} (a(x_i) - y_i)^2}$$

RMSE and MSE are good for comparing two different models, for quality control when training models, but does not allow us to derive conclusions on how good our model solves a particular task.

MSE = 10, would be an awful result if the target variable y would take values (0,1). However it would be a great result if y would take values (10000, 100000).

Coefficient of determination R^2 .

Essentially, the coefficient of determination R^2 is a normalized MSE, the closer it is to 1 the better the model is at making predictions, and the closer it is to 0, the closer it is in quality to having a constant prediction.

$$R^2(a, X) = 1 - \frac{\sum_{i=1}^{\ell} (a(x_i) - y_i)^2}{\sum_{i=1}^{\ell} (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{\ell} \sum_{i=1}^{\ell} y_i$$

Mean absolute error (MAE).

Suppose we've replaced the squared difference with an absolute value.

$$L(y, a) = |a - y|$$

The error function resulting from this change is the mean absolute error (MAE)

$$\text{MAE}(a, X) := \frac{1}{\ell} \sum_{i=1}^{\ell} |a(x_i) - y_i|$$

The mean absolute error (MAE) is not differentiable but is less sensitive to outliers that might ruin a model in more outlier sensitive error functions such as MSE, RMSE, and coefficient of determination.

Let's show why the mean absolute error (MAE) is less sensitive to outliers.

Suppose all ℓ objects from $\mathbb{X}$ have the same feature descriptions, but different values of the target variable $y_1, \dots, y_\ell$. In this case the model has to give the same answer on all these $\mathbb{X}$. If we picked MSE as the error function, we get the following problem[^]:

$$\frac{1}{\ell} \sum_{i=1}^{\ell} (a(x_i) - y_i)^2 \rightarrow \min_a$$

The minimum would be obtained on the average of all the answers:

$$\frac{1}{\ell} \sum_{i=1}^{\ell} 2(a - y_i) = \frac{2}{\ell} \sum_{i=1}^{\ell} (a - y_i) \Rightarrow \frac{2}{\ell} \sum_{i=1}^{\ell} (a - y_i) = 0 \Rightarrow \sum_{i=1}^{\ell} (a - y_i) = 0 \Rightarrow a = \frac{1}{\ell} \sum_{i=1}^{\ell} y_i$$

$$a_{\text{MSE}}^* = \frac{1}{\ell} \sum_{i=1}^{\ell} y_i = \bar{y}$$

Thus, if even one of the answers is different from the others (is an outlier), the average would be greatly skewed in its direction.

Now image the exact same situation but with an MAE function.

$$\text{MAE}(a, X) := \frac{1}{\ell} \sum_{i=1}^{\ell} |a(x_i) - y_i| \rightarrow \min_a$$

$$\frac{1}{\ell} \sum_{i=1}^{\ell} |a(x_i) - y_i| \underset{\ell=m+k}{=} \frac{1}{\ell} \left(\sum_{i=1}^m (a - y_i) + \sum_{i=m+1}^k (y_i - a) \right) \underset{\text{diff}}{=} \frac{1}{\ell} (\# \text{ of } \leq (a) - (\ell - \# \text{ of } \leq (a)))$$

Which can be rewritten as:

$\frac{1}{\ell} (2 \# \leq (a) - \ell)$ which when differentiable would be minimized at the change of the derivative's sign.

$$a_{\text{MAE}}^* = \text{median}(y_1, \dots, y_\ell)$$

Huber loss

If we attempt to unite the absolute loss function and the quadratic loss function we would obtain the Huber loss function.

$$L_\delta(y, a) := \begin{cases} \frac{1}{2}(y - a)^2, & |y - a| < \delta \\ \delta(|y - a| - \frac{1}{2}\delta), & |y - a| \geq \delta \end{cases}$$

The key aspect of this error function is the δ parameter that regulates what we consider to be an outlier. The smaller this parameter δ is, the error function will act as a quadratic one only when the difference between the answer and the target variable is near 0 ($(a - y) \approx 0$). On the other hand, if the δ is greater, then the error function will act as a quadratic one even with a greater different.

Essentially, $\begin{cases} \delta \rightarrow 0, & \text{Huber is absolute} \\ \delta \rightarrow \infty, & \text{Huber is quadratic} \end{cases}$

One must pick the parameter δ for each individual case.

As all things though, the Huber loss function has a large problem, that being that its derivative has breaks.

Log-Cosh

The Log-Cosh function accounts for the Huber loss's problem of derivative's breaks.

$$L(y, a) := \log \cosh(a - y)$$

Similarly to the Huber error function, for small deviations it would act as a quadratic error, and for larger deviations the Log-Cosh error function would act as a linear one.

Mean squared logarithmic error (MSLE).

$$L(y, a) := (\log(a + 1) - \log(y + 1))^2$$

This particular error function is used when we are faced with a task that has a nonnegative target variable and nonnegative results of the model.

By applying a log onto the results, we are punishing the model for both overestimating and underestimating the result (where underestimation is typically penalized more heavily in relative terms).

Mean absolute percentage error (MAPE)

Essentially, measures the average absolute percent error between predictions and actual values. Furthermore, they are best when we are comparing with variables with different scales. Here is a standard example of the mean absolute percentage error (MAPE):

$$L(y, a) := \left| \frac{y - a}{y} \right|$$

As everything, it has certain drawbacks:

1. The mean absolute percentage error (MAPE) becomes undefined if values are close to zero, thus, **it penalizes positive errors** ($a > y$) **more heavily than negative errors** ($a < y$).
2. 1. MAPE has an asymmetry problem: since the denominator is the actual value y , it penalizes positive errors (overestimation) more heavily than negative errors (underestimation) in terms of the percentage calculated.

Symmetric mean absolute percentage error (SMAPE).

Meant to address the asymmetry problem of the mean absolute percentage error (MAPE).

Essentially, the symmetric squared mean absolute percentage error (SMAPE) is an accuracy based measure based on percentage (or relative) errors. It is meant to treat over and under forecasts in a relative fashion instead of an absolute way.

$$L(y, a) := \frac{\frac{|y - a|}{|y| + |a|}}{2}$$

Quantile cost function

Often the cost of overestimating or underestimating the prediction is not equal, and one of them is larger. We wish to account for this possibility, and thus we use the quantile loss (MSE). In such cases we are presented with an classic optimization problem.

We introduce a parameter $\tau \in [0, 1]$, which will determine the proportion of importance for the overestimation and underestimation the prediction. Suppose that the larger τ is, the larger the penalty for underestimating the prediction will be.

$$Q(a, X^\ell) = \sum_{i=1}^{\ell} p_{\tau}(y_i - a(x_i)), \quad p_{\tau}(z) := (\tau - 1)[z < 0]z + \tau[z \geq 0]z$$

3. Linear Regression

3.1. Overfitting

Overfitting occurs when a model performs exceptionally well on the training set, but fails to produce the same quality of results when presented with new data.

This problem can be addressed via:

1. Using a narrower class of models.
2. Punish the model for overcomplicating the final model.
3. Perform **regularization**.

3.2. Estimating the quality of models

In order to check ourselves the model we've trained for possible overfitting, we can divide the data we have into a classic training set X and a **holdout-set** (отложенная выборка), which we will use to test the data.

As all things, dividing the data into a training and a holdout (testing) set brings about certain difficulties. We do not know what kind of model quality we would have had if we'd used the full dataset as our training set.

Crossvalidation.

We break the data set into k sections $X_1, \dots, X_k$ of roughly the same size. Then we are to train k models $a_1(x), \dots, a_k(x)$ where the i -th model is trained on all objects x from all models excluding the i -th model.

The quality of each model is evaluated on that i -th section that was not used in its training, and the results are averaged.

$$CV := \frac{1}{k} \sum_{i=1}^k Q(a_i(x), X_i)$$

Suppose that we've performed the crossvalidation and now we need to obtain the final version of the model. This could be done in the following ways.

1. We perform crossvalidation in order to obtain the best possible model from the k sections. Once it is selected, we train all of the data on that final option with the assumption that we've minimized the risk of overfitting.
2. If we are not able to train the final model, be it because of size constraints, or other features, we can build a model composition $a_1(x), \dots, a_k(x)$ obtained during the crossvalidation. Here, in the case of regressions, we can interpret the composition of models as the averaging of the model's predictions.

3.3. Training a linear regression

Linear regressions are often trained using an mean squared error function (MSE). In such cases we are presented with an classic optimization problem.

$\frac{1}{\ell} \sum_{i=1}^{\ell} (\langle w, x_i \rangle - y_i^2) \rightarrow \min_w$, which if rewritten in a matrix form: $\frac{1}{\ell} \|Xw - y\|^2 \rightarrow \min_w$ where we use the Euclidean norm.

If we solve the optimization problem the final result would be: $w = (X^T X)^{-1} X^T y$

Being able to easily find the optimal vector of weights is a boon to the linear regression over other models, nevertheless, as all things, it is not always applicable for a range of reasons.

1. **Inverting a matrix** is a complex operation with cubic complexity relative to the number of features (d).
2. The matrix $X^T X$ could either be singular (uninvertible, determinant is zero) or ill-conditioned (incredibly close to a singular). To solve this problem one would have to perform **regularization**, which will be discussed further.

4. Gradient descent

Optimization problems as the ones we saw with the MSE function can be solved iteratively using gradient methods.

4.1. Gradient and its characteristics

Gradient of a function.

The gradient of a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is the vector of all its partial derivatives:

$$\nabla f(x_1, \dots, x_d) = \left(\frac{\delta f}{\delta x_j} \right)_{j=1}^d$$

At any given point, the gradient points in the direction of the greatest rate of increase of the function, whilst the antigradient ($-\nabla$) points in the direction of the greatest decrease of the function.

4.2. Gradient descent

Essentially, the gradient points in the direction of the greatest rate of increase of the function. The antigradient does the same in the direction of the greatest rate of decrease of the function.

We ought to start on a certain point, calculate the antigradient in said point and take a step towards it, recalculate the antigradient, and take another step.

Gradient descent.

More formally, the gradient descent is the repetition of the following:

$w^{(k)} = w^{(k-1)} - \eta_k \nabla Q(w^{(k-1)})$, where $Q(w)$ is the value of the cost function for the parameters w and η_k is the length of the step required, iterated until convergence is reached.

It is important to carefully pick the **step** η_k (also known as the learning rate). It could be a constant, but if the constant step is too large, we run a risk of jumping over the minimum, whilst if the constant step is too small, then finding the minimum would take far too many iterations.

Here is an often used default step, where we reduce the step as we approach the minimum.

$$\eta_k = \frac{1}{k}$$

Here is the step formula used in vowpal wabbit.

$$\eta_k = \lambda \left(\frac{s_0}{s_0 + k} \right)^p, \lambda, s_0, p \text{ are just some parameters.}$$

When do we stop the gradient descent?

We can halt the iterations when the gradient starts to approach zero ($\|\nabla Q(w^{(k-1)})\| < \varepsilon$) or when the changes in the weight vector cause by each step become way too small ($\|w^{(k)} - w^{(k-1)}\| < \varepsilon$)

Another good alternative is to watch for values of a cost function on a holdout-set and halt when the value stops dropping.

4.3. Evaluating a gradient

As a rule of thumb, the cost function $Q(w)$ is presented as a sum of ℓ functions:

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} q_i(w), \text{ where } q_i(w) \text{ is the error on certain object } x$$

Recalling a previously presented method of a gradient descent ($w^{(k)} = w^{(k-1)} - \eta_k \nabla Q(w^{(k-1)})$), we can see that on each iteration we would need to calculate the gradient of the whole sum.

$$\nabla_w Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \nabla q_i(w)$$

The primary issue with standard gradient descent is the necessity of calculating the full gradient across all ℓ objects at every step. This becomes computationally problematic for massive datasets.

This issue could be mitigated if we accept that we don't always need the **precise** value of the gradient, and we could do with approximations, which we will review in this section.

4.3.1. Stochastic gradient descent

The **Stochastic gradient descent (SGD)** addresses the primary issue of the gradient descent by estimating the gradient using a single randomly selected object i_k from the functional.

$$w^{(k)} = w^{(k-1)} - \eta_k \nabla q_{i_k}(w^{(k-1)})$$

Because the gradient of a single object may not be zero at the global minimum, SGD may deviate from the optimum even when close to it. To ensure convergence for convex functions, the step size η_k must satisfy the **Robbins-Monro conditions**:

$$\sum_{k=1}^{\infty} \eta_k = \infty, \quad \sum_{k=1}^{\infty} \eta_k^2 < \infty$$

A major advantage of SGD is its memory efficiency; since only one object is required per step, models can be trained on datasets that do not fit in RAM.

The accuracy of the gradient can be increased by using several objects instead of a single one.

$\nabla_w Q(w) \approx \frac{1}{n} \sum_{j=1}^n \nabla_w q_{i_{kj}}(w)$, where i_{kj} are randomly selected indexes of objects from the cost function, and n is the method's parameter, that acts as the size of the object batch for one gradient step.

Essentially, with this modification we obtain the **mini-batch gradient descent**, that acts as a compromise where the gradient is estimated using n objects, providing a more stable direction than a single object while remaining more efficient than the full gradient.

4.3.2. SAG method

The **Stochastic average gradient (SAG)** method combines the low complexity of the Stochastic gradient descent (SGD) iterations with the high convergence rate of full gradient descent. It stores the last calculated gradients for every object in the training set.

$$z_i^{(k)} = \begin{cases} \nabla q_i(w^{(k-1)}) & , i = i_k \\ z_i^{(k-1)} & \text{otherwise} \end{cases}, \quad \text{where } w^{(k)} = w^{(k-1)} - \eta_k \frac{1}{\ell} \sum_{i=1}^{\ell} z_i^{(k)}$$

While only one gradient is updated per step, the weight update uses the average of all stored gradients. This allows SAG to achieve a convergence rate of $O\left(\frac{1}{k}\right)$ for convex functions, matching full gradient descent.

4.3.3. Other approaches

Gradient estimation can also be performed without calculating analytical derivatives. For example, one can sample a random vector u from a unit sphere and use the function's value in that direction to approximate the gradient:

$$\nabla_w Q(w) \approx Q(w + \varepsilon u)u$$

Alternatively, for complex architectures like deep neural networks, one can train a model to predict the gradient based on current parameters, known as synthetic gradients.

4.4. Modifications of the gradient descent

4.4.1. Momentum (Метод инерции)

Used in cases where level lines are highly stretched, the anti-gradient direction often changes sharply, causing noise and slow optimization. **Momentum** addresses this by averaging the anti-gradients from previous steps using a **momentum vector** h_k :

$$\begin{aligned}h_0 &= 0 \\h_k &= ah_{k-1} + \eta_k \nabla_w Q(w^{(k-1)}) \\w^{(k)} &= w^{(k-1)} - h_k\end{aligned}$$

The parameter a determines the decay rate of past gradients. If a gradient coordinate consistently points in the same direction, the momentum vector grows, accelerating the descent.

4.4.2. AdaGrad and RMSprop

The gradient descent is incredibly sensitive to the size of the gradient step η . There are no methods for predetermining the correct step η one has to pick. **AdaGrad** introduces a unique step length for each parameter component, which decreases based on the length of previous steps.

$$\begin{aligned}G_{kj} &= G_{k-1,j} + (\nabla_w Q(w^{(k-1)}))_j^2 \\w_j^{(k)} &= w_j^{(k-1)} - \frac{\eta}{\sqrt{G_{kj} + \varepsilon}} (\nabla_w Q(w^{(k-1)}))_j\end{aligned}$$

This is effective for sparse tasks. However, because G_{kj} grows monotonically, steps eventually become too small to reach the minimum. **RMSprop** solves this by using exponential decay for gradient accumulation:

$$G_{kj} = aG_{k-1,j} + (1 - a)(\nabla_w Q(w^{(k-1)}))_j^2$$

4.4.3. Adam

Adam (Adaptive Moment Estimation) combines the benefits of both Momentum and RMSprop. It accumulates past gradients to avoid oscillations while maintaining an adaptive step size for every individual parameter.

$$\text{Adam} \approx \text{Momentum} + \text{RMSprop}$$

5. Linear classification

In this section we will review a binary linear classification, that occurs when the target set $\mathbb{Y} = \{-1, +1\}$. The model predicts the class based on the sign of a linear combination of features.

Suppose that $\mathbb{X} = \mathbb{R}^d$ is the object set, $\mathbb{Y} = \{-1, +1\}$ is the target set, $X = \{(x_i, y_i)\}_{i=1}^{\ell}$ is the training set, targets will be referred to as positive and negative.

$a(x) = \text{sign}(\langle w, x \rangle + w_0)$ where $w \in \mathbb{R}^d$ is the weight vector, and $w_0 \in \mathbb{R}$ the bias.

If not otherwise stated, we can assume that there is a constant $x_{d+1} = 1$, and as a result:

$$a(x) = \text{sign} \langle w, x \rangle$$

One must also consider that we may encounter the unlikely case of $\text{sign}(z) = 0$, which we will consider to be the model refusing to classify the object.

5.1. Training linear classifiers

A linear regression problem has a continuum of potential answers, it would be odd to demand a complete match between the model and the true answers. It is far more reasonable to demand a certain “closeness” between them.

Furthermore, attempts to match perfectly to the training set, as we’ve learned before, increase the risk of overfitting.

The initial cost function for the (binary) linear classifier has one wrong and one right answer. We wish to see as many correct answers as possible. The resulting cost function is the rate of correct answers, which called **accuracy**.

$$Q(a, X) = \frac{1}{\ell} \sum_{i=1}^{\ell} [a(x_i) = y_i]$$

Nevertheless, it would be easier for the challenged ahead to solve minimization problems, and thus, we will use the rate of incorrect answers.

$$Q(a, X) = \frac{1}{\ell} \sum_{i=1}^{\ell} [a(x_i) \neq y_i] = \frac{1}{\ell} \sum_{i=1}^{\ell} [\text{sign} \langle w, x_i \rangle \neq y_i] \rightarrow \min_w$$

This cost function is discrete regarding the model’s weights, and thus looking for the minimum using gradient methods is impossible, and furthermore cost function may have several minimum points, complicating the potential solution further. As a result we will attempt to transform the problem into a minimization problem of a smooth (both continuous and differentiable limit) cost function.

5.1.1. Margins (Отступы)

The **margin** M_i for an object is defined as the product of its true label and the model’s scalar output.

$$M_i = y_i \langle w, x_i \rangle$$

The sign of the margin indicates correctness: a positive margin ($M_i > 0$) means the classifier's answer is correct, while a negative margin ($M_i < 0$) indicates an error. The absolute value $|M_i|$ characterizes the model's confidence. It can be obtained by slightly transforming our formula of **accuracy**.

$$Q(a, X) = \frac{1}{\ell} \sum_{i=1}^{\ell} [a(x_i) = y_i] = \frac{1}{\ell} \sum_{i=1}^{\ell} \left[\underbrace{y_i \langle w, x_i \rangle}_{M_i} < 0 \right] \rightarrow \min_w$$

5.1.2. Upper Bounds (Верхние оценки)

The cost function estimates the error of the model on the x object with the help of a fall function $L(X) = [M < 0]$, where the argument M is the margin.

$$Q(a, X) \leq \frac{1}{\ell} \sum_{i=1}^{\ell} \hat{L}(M_i) \rightarrow \min_w$$

If the smooth upper bound is minimized toward zero, the actual error rate will also decrease towards zero.

Some examples of upper bounds can be seen here (we will discuss them fully at a later date):

1. Logistic loss: $\log(1 + e^{-M})$. Used to estimate class probabilities in Logistic Regression.
2. Hinge loss (SVM): $\max(0, 1 - M)$. The foundation of the Support Vector Machine (SVM).
3. Perceptron loss: $\max(0, -M)$. Employed in the Rosenblatt Perceptron algorithm.
4. Exponential loss: e^{-M} . Frequently used in boosting algorithms (sequential learning of 'weak' models, each fixin the mistakes of the previous).
5. Sigmoid loss: $\frac{2}{1 + e^M}$. Provides a smooth, bounded approximation to the standard threshold error function.

5.2. Quality metrics of classification

Whilst we have previously transformed the problem of training a linear classifier into a problem of minimization of a smooth cost function, there is a need in developing different ways of measuring the quality of the trained classifier.

For the following section we will suppose that:

Linear classifier: $a(x) = \text{sign}(b(x) - t) = 2[b(x) > t] - 1$ which will take on this view if we presuppose that $b(x) = \langle w, x \rangle$ and $t = 0$.

5.2.1. Fraction of correct answers (accuracy)

Accuracy is the simplest measure of quality, representing the proportion of correct predictions.

$$\text{accuracy}(a, X) = \frac{1}{\ell} \sum_{i=1}^{\ell} [a(x_i) = y_i]$$

Accuracy is flawed in imbalanced datasets. For instance, if 95% of objects belong to one class, a trivial model always predicting that class achieves 95% accuracy without learning any patterns.

5.2.2. Error matrix

The **Error Matrix** breaks down predictions into four categories based on the combination of true and predicted classes.

	$y = 1$	$y = -1$
$a(x) = 1$	True Positive (TP)	False Positive (FP)
$a(x) = -1$	False Negative (FN)	True Negative (TN)

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{FN} + \text{TN}}$$

Quality metric	Description
Precision	Measures the quality of positive predictions.
Recall	Measures the model's ability to find all positive objects.
F-measure	Balanced metric that acts as a smoothed minimum of precision and recall.
R-precision (breakeven point)	The precision calculated at the specific threshold where precision and recall are equal.
Average precision	Used when objects need to be sorted by confidence.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

$$\text{R} = \text{precision}(\text{sign } b(x) - t^*), \quad t^* = \arg \min_t |\text{precision}(\text{sign } (b(x) - t))|$$

$$\text{AP} = \frac{1}{\ell_+} \sum_{k=1}^{\ell} [y_k = 1] \text{precision@}k, \quad b(x) \text{ is } a(x) \in \{\mathbb{R} \setminus \mathbb{Z}\}, \text{ where:}$$

y_k is the k-th ordered target variable, ℓ_+ the amount of positive objects in the training set, $\text{precision@}k$ is the accuracy across the first k objects in the object list

5.2.2.1. Relation between precision, fullness, and fraction of correct answers

Essentially, high requirements for both precision and fullness in imbalanced tasks force an extremely high accuracy. For example, in a set of 1,000,000 objects where only 1% are positive, requiring 90% precision and 90% fullness demands an accuracy of 99.8%.

5.2.2.2. Lift

Lift represents the improvement in the concentration of positive objects compared to a random selection.

$$\text{lift} = \frac{\text{Precision}}{\frac{\text{TP} + \text{FN}}{\ell}}$$

5.2.3. Area Under Curve (AUC)

AUC evaluates a family of classifiers across all possible thresholds t .

5.2.3.1. Index Gini

The **Gini Index** is proportional to AUC and is frequently used in credit scoring.

$$\text{Gini} = 2 \cdot \text{AUC} - 1$$

5.2.3.2. Sensitivity to the relation of classes

AUC-ROC is sensitive to class imbalance when the positive class is significantly smaller. A poor model can still have a high AUC-ROC because it measures False Positives relative to the total number of negative objects. In tasks like identifying math articles (100 out of 1,000,100), a useless model can achieve an AUC-ROC of 0.95.

5.2.3.3. Precision-Recall curve

The **Precision-Recall (PR) curve** is often better for imbalanced classes. It plots Fullness on the x-axis and Precision on the y-axis. The area under this curve (**AUC-PR**) provides a more adequate quality assessment in such cases.

$$\text{Average Precision (AP)} = \frac{1}{\ell_+} \sum_{\{k=1\}}^{\ell} [y_{\{(k)\}} = 1] \text{ precision@}k$$

AP approximates the area under the PR curve

5.3. Log-regression

5.3.1. Estimating probabilities

5.3.2. Model realism and Logistic loss

5.3.3. Logistic regression

5.4. Support vector machine

5.4.1. Case 1

5.4.2. Case 2

5.4.3. Case 3

—

6. Multi-class classification and categorical features

6.1. Multi-class classification

6.1.1.

6.1.1.1. One-versus-all

6.1.1.2. All-versus-all

6.1.2. Multi-class logistic regression

6.1.3. Multi-class support vector machine (SVM)

6.1.4. Quality metrics

6.2. Classification with overlapping classes

6.2.1. Binary relevance (Independent classifier)

6.3. Categorical features

6.4. Seminar

We will look at the conditions a classifier must follow so that it could be used

6.4.1. Prediction of probabilities.

If we recall RUC-AUC, this approach, as all, has some problems.

$$\text{ROC-AUC} \approx 0.8 \mid \overbrace{0.9 \ 0.7 \ 0.4 \ 0.2 \ 0.1}^{+1 \ +1 \ +1 \ +1 \ +1}$$

$$\text{ROC-AUC} \approx 0.8 \mid \overbrace{9 \ 7 \ 4 \ 2 \ 1}^{+1+1+1+1+1}$$

$$\text{ROC-AUC} \approx 0.8 \mid \overbrace{-1 \ -3 \ -6 \ -4 \ -9}^{+1 \ +1 \ +1 \ +1 \ +1}$$

Calibrated model.

Suppose we are given viborka X . The model $\hat{p}(x)$ — is calibrated if:
predicted probability

$\forall$ the objects $x' \in X, \forall \hat{p}(y = 1 \mid x') = k, k \in [0, 1]$, is equal to $k \cdot 100\%$ percent really belong to the target class.

The solution of this task depends on the Loss function $L(\cdot)$:

$\arg \min(\hat{p} \in R) \mathbb{E}[L(y, \hat{p}(x) \mid x)] = p(y = +1 \mid x)$, where $p(y = +1 \mid x)$ is the истинная вероятность

6.4.2. Task 1.

Show that the MSE $L(y, \hat{p}) = ([y = +1] - \hat{p})^2$ allows to predict correct probabilities.

$$\text{MSE} = ([y = +1] - \hat{p})^2, \mathbb{E}(\text{MSE}(y, \hat{p} \mid x)) = \overbrace{p(1 - \hat{p})^2}^{y=+1} + \overbrace{(1 - p) \cdot \hat{p}^2}^{y \neq +1}$$

take the derivative from page 2 of the 6th seminar document

6.4.3. Task 2.

Show that the MSE $L(y, \hat{p}) = |[y = +1] - \hat{p}|, \hat{p} \in [0, 1]$ allows to predict correct probabilities.

$$\mathbb{E}(\text{MAE}) = p \cdot |1 - \hat{p}| + (1 - p) \cdot |\hat{p}| = p|1 - \hat{p}| + (1 - p) \cdot \hat{p}$$

$$\frac{\delta(\mathbb{E}(\text{MSE}))}{\delta(\hat{p})} = -p + 1 - p = 1 - 2p \Rightarrow 0 = 1 - 2p$$

We can now check two different resulting cases:

1. $p = \frac{1}{2} \forall \hat{p}$ And as a result, the classifier is unable to predict the correct probability in point x .

2. $p \neq \frac{1}{2}$ And as a result the $(0,1)$ interval does not contain critical points,

6.4.4. Calibration of probabilities .

In order to obtain a calibrated model, we can build a new model over the old model, that will take a probability (generated by the original classifier ($\hat{p}(x)$)) as an input, and transforms it into the

6.4.5. Platt Scaling (Platt calibration)

$$P(y = 1|x) = \frac{1}{1 + \exp(af(x) + b)}$$

Example:

Suppose that $\hat{p} = 0.7 \rightarrow p = 0.7$ and that the probabilities are $0, \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, 1$.

It's all great when they are so neatly laid out, but if we are given something like 0.3845911... what should we do?

We offered to build a distogram, and then calculate the amount that would be present in a certain class.

From the distogram we are able to build a probabilities plot (calibration plot) that will be located in the axis $y = \text{observations}$, $x = \text{probability}$

Then we consider that the best option for the calibration plot is a 45 degree straight line.

6.4.6. Isotonic regression

6.4.7. Quantile regression

6.4.8. Beta calibration

6.4.9. Multi-class calibration